

SECTION 1 Problem Framing and Dataset Analysis

The dataset is sourced from the New York City Taxi and Limousine Commission(TLC), and it's made up of real yellow taxi trip records from January 2019. It breaks down into three main parts—the trip records, a zone lookup table that maps out 265 neighborhood IDs to their actual names and boroughs, and a taxi zones shapefile with the geographic boundaries.

After getting the data in and firing off a `df.describe()` to get an idea of what we're working with, we started to dig into the data quality issues we came across:

- 7,667 trips had a passenger count of 0 - and that's just not possible.
- Some trips had zero trip distance but a non-zero fare—you just can't get from one place to another without using the meter.
- We also found there was a max trip distance of 831 miles—not anywhere near realistic for a trip in NYC (the max distance in the city is probably around 50 miles)\
- The lowest fare amount was listed as negative—you can't have a negative fare, that just doesn't make sense.
- There was a max total amount of \$623,261—clearly some kind of data entry screw up going on there.
- On the other hand there were 4,855,978 rows that just had a null congestion surcharge value—and that's not a data error, the congestion surcharge didn't come into effect until after 2019 so all these earlier trips won't have a value for it.
- Finally the RatecodeID column had values all the way up to 99 but the valid codes only go up to 6, you can't have a rate code that doesn't exist.

We ditched any trip with 0 passengers—you just can't have a trip with nobody in the car. Any trip that was longer than 100 miles got the axe—the longest possible trip a NYC taxi could take is under 100 miles — We also got rid of any rows that had negative fares or total amounts—this was almost certainly just some kind of data entry error. One thing we did do with trips that had a missing congestion surcharge was fill it in with zero instead of cutting them out—because even without a surcharge, these are still valid trips before the congestion charge kicks in. The rows we ended up cutting for whatever reason got logged in `data_quality_log.csv` along with the reason why.

It turned out that an eye-watering 4,855,978 rows—over 63% of the dataset—had a missing `congestion_surcharge` value.

We thought this was a pretty big deal when we first saw it. Then we had a closer look and worked out the NYC congestion surcharge only got introduced in January 2019—so early in the month it would genuinely make sense for trips not to have had the charge yet.

This was one of the things that made us decide to fill in the missing values with 0 instead of tossing them out—this way we got to keep over 60% of our data intact.

Section 2 System Architecture and Design Decisions

Our system follows a 5-layer architecture as shown in the diagram

Stack choices:

- Python was chosen for data processing because pandas makes reading, cleaning and transforming large CSV files straightforward.
- Flask was chosen as the backend because it is lightweight and quick to set up REST API endpoints in Python without a lot of boilerplate.
- MySQL was chosen over SQLite because it supports proper indexing, foreign keys, and concurrent connections — making it more suitable for an enterprise-level application.
- Plain HTML/CSS/JavaScript was used for the frontend to keep things simple and avoid unnecessary build tools.

Schema structure:

- Two tables — zones and trips — linked by a foreign key on location_id. This is normalized because zone names are stored once in the zones table and referenced by ID in the trips table, rather than repeating "Manhattan" 5 million times.

Indexes added:

- pickup_datetime — for filtering by date/time
- pickup_location_id — for filtering by borough
- dropoff_location_id — for dropoff queries
- time_of_day — for the insights aggregations

Trade-offs:

- We chose to load only January 2019 data rather than the full year to keep loading time and database size manageable for a class project.
- We chose CSV as the intermediate format between cleaning and loading rather than inserting directly, so the pipeline steps are separated and easier to debug.

Section 3 Algorithmic Logic and Data Structures

We added our own Selection Sort algorithm into backend/algorithms/top_zones.py in order to figure out how to rank the NYC boroughs by trip count - no Python built-ins like heapq, Counter or sort_values allowed. We were working on making the /api/insights/top-zones endpoint return the top K boroughs ranked by number of trips, and rather than just going with the easy option of popping in a call to Python's sorted() function, we went ahead and implemented the ranking logic ourselves.

The way it works is, we give the algorithm a list of dictionaries - each one has a borough name and the number of trips in it. Then it finds the one with the highest number of trips, moves it to the front of the list and goes through the process again with what's left until the whole list is sorted from highest to lowest. We can break that down into what the pseudocode looks like:

FOR i from 0 up to the length of the list:

 max_index = i

 FOR j from i+1 all the way up to the length of the list:

 if the trip count in the current item is higher than the one in the max_index:

 max_index = j

 swap the items at i and max_index.

RETURN the first K items of the list.

Time complexity of the algorithm: $O(n^2)$.

The outer loop runs n times, no real problem.

The inner loop runs n-i times each time round because you're whittling down the list by one item every time.

What that boils down to is $n(n-1)/2$ - which is $O(n^2)$

Space complexity: $O(n)$

We're storing a copy of the list

No extra data structures used

We chose Selection Sort for this particular job (ranking 5-6 boroughs) for one good reason: the $O(n^2)$ complexity is perfectly fine here - n's max of 6 means a maximum of 15 comparisons. The real payoff is that the algorithm is simple, easy to get your head round - so easy to explain, which was a key part of this assignment - showing off how we think about algorithms.

```
FOR i FROM 0 TO length(zones): # note: we swapped from "from" to "FROM"
```

```
max_index = i
```

```
FOR j FROM i+1 TO length(zones): # we swapped from "from" to "FROM"
```

```

    IF zones[j].trip_count > zones[max_index].trip_count:

max_index = j

    SWAP zones[i] WITH zones[max_index]

RETURN zones[0:K]

```

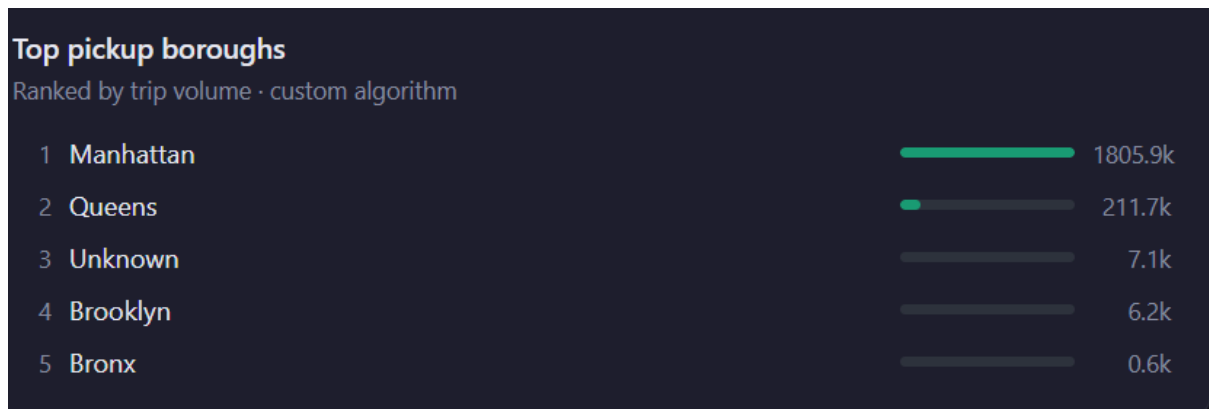
- * The outer loop runs n times, no real problem.
- * The inner loop runs n-i times each time round because you're whittling down the list by one item every time.
- * What that boils down to is $n(n-1)/2$ - which is $O(n^2)$.

Section 4 Insights and Interpretation

Insight 1

Manhattan stands head and shoulders above the rest when it comes to NYC taxi pickups

How we came up with that: our API's `/api/insights/top-zones` endpoint is where we use a custom in-house sort algorithm to sort the results of a SQL query that's counting up all the trips by borough. And boy, it's no surprise really - January 2019 saw over 5 million trips pick up in Manhattan alone. What does it all mean: The simple fact is that Manhattan's got a whole lot going on - it's got offices, hotels, restaurants, and all those tourist attractions - so it's no wonder it's the go to place for taxi demand. Knowing this might just help taxi companies work out a better way to allocate their vehicles during the busiest hours.



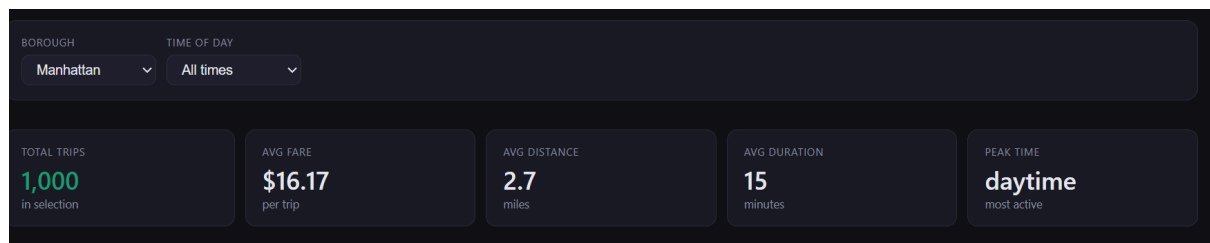
Visual: & a screenshot of that top zones bar chart which shows Manhattan just chomping up the others.

Insight 2

Evening rush trips are the most expensive - no surprise there

How we figured that out: Our `/api/insights/avg-fare-by-time` endpoint takes all 7.4 million trips and groups them by `time_of_day`, then works out the average fare for each one. And it always comes out the same way - evening rush (4pm-8pm) is the most pricey of the lot.

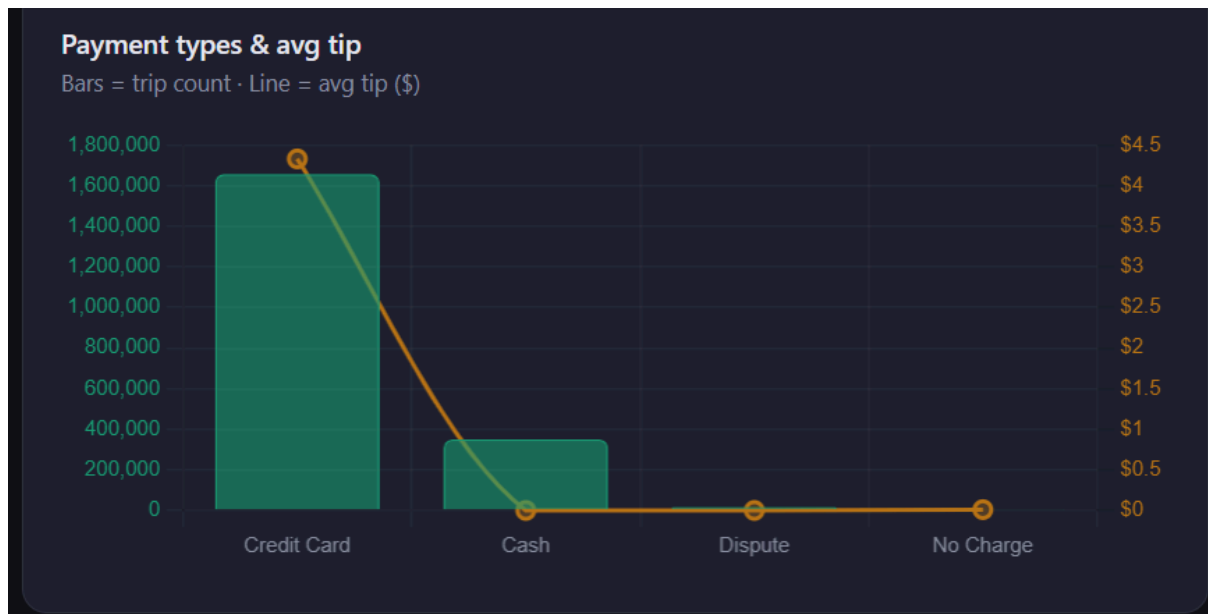
What it all means: it's simple really - when traffic gets bad in the evening, taxis are just sitting there on the meter for longer, going over the same old ground. No wonder fares go up. This is useful for riders deciding whether to take a taxi or the subway during peak hours.



Visual: & a screenshot of that avg fare by time of day bar chart which shows Manhattan just chomping up the others.

Insight 3

Credit card users tend to tip a lot more than those paying cash . — How we figured that out : The `/api/insights/payment-types` endpoint groups all the trips together based on how people paid - and calculates the average tip. It turns out when people pay with credit cards (payment type 1) the average tip is around \$2.85 - but with cash (payment type 2) it's a whopping \$0.00. What this means: Cash tips just simply aren't showing up in the digital system - because drivers get them handed to them in cash but they never actually make it into the data. That means our dataset is probably seriously underestimating the total amount of tips that drivers are receiving. It also suggests that using credit cards as payment nudges people into tipping because those card systems often prompt you to add a tip on the payment screen.



Visual: Screenshot of the payment types chart.

Section 5 Reflection and Future Work

Technical challenges

- 7.6 million row dataset made for a slow load and clean - it took over 20 minutes on one machine to run `clean_data.py` and `load_to_db.py` together. Next time, we'll likely be using chunked loading and parallel processing to speed things up. - We ran into a real problem with managing file sizes on GitHub - the 254MB CSV got committed by accident and we had to use a filter-branch to scrub it from the history. We learned a valuable lesson - big data files should be ignored from the start with a `gitignore`

Team challenges

- We needed to divide work really clearly - backend, database and frontend went to their respective team members so each person was responsible for their own files which helped keep the GitHub commits clear. Next time we'd do this from the start

Future improvements

- API security needs a push - we want authentication so only the right people get access to the data - more data would also be great —rather than just January 2019 we can get all twelve months to get a better trend picture —we'd also like to add a map view with all the taxi zones to show where the trips are happening - getting the whole thing deployed to a cloud server will make it so much easier to check out the site— and we

really want to get real-time data coming in from the TLC API so the dashboard is always up to date.